

本研究の一部は JSPS 科研費 29K06372 の助成を受けています。

帯広畜産大学データサイエンス 演習における取り組み

齊藤 準

帯広畜産大学 人間科学研究部門 / 農学情報基盤センター

CloudWeek2026@Hokkaido University

2026-09-02

自己紹介

- 齊藤 準 / Jun SAITO, 専門＝素粒子理論, 物理教育, 学習分析
- 現職＝帯広畜産大学 人間科学研究部門・農学情報基盤センター 准教授
 - 農学＝統計プログラミングが重要
 - データサイエンス教育
(前任者から引き継ぎ) SAS, Octave → (改訂) R, Python
 - MDASH (リテラシー, 応用基礎) WG座長, 三大学DS教育WG員
- Jupyter の活用
 - 学生PC, 実習室PCにAnacondaでインストール → 構築・管理が困難
 - 2022年: 藤原先生のご支援でJDCat分析ツールを活用
 - 2025年: 大江先生のご支援でMCJ-CloudHubを活用

概要

■ 試運用環境での活用実践（2025年度）

- NII 提供の MCJ-CloudHub 環境を試運用
- Python & R, 自動採点, ChatGPT API 等の活用

■ 構築・本運用に向けて（2026年度）

- AWS での構築練習
- mdx, オンプレミス（GPUマシン）での構築検討

試運用環境での活用実践 (2025年度)

背景

■ MDASH 申請・認定に伴うカリキュラム改訂・強化

- 科目新設「データサイエンス演習」
- 大人数による安定・高効率なプログラミング環境の必要性

■ MCJ-CloudHub との出会い

- 2022年から JDCat 分析ツールの活用による授業実践
- 学術情報基盤オープンフォーラム2025に登壇
→MCJ-CloudHub による教材配布, 課題提出, 自動採点等の可能性を知る

実践

■ データサイエンス演習 (データとAI)

- 2025年度新設，第3クォーター（10月～12月）全8回
- 1年生97名，教員 1名（TASAなし）
- 全課題で Python と R を併用
- データエンジニアリングの基礎，AIの利用（MDASH応用基礎レベル）

第1回	Python / Rプログラミングの基礎
第2回	さまざまなデータとその取得
第3回	データの集計・要約
第4回	データの可視化
第5回	前処理・加工，データフレーム，データベース
第6回	AI活用：音声・画像（深層学習，認識技術）
第7回	AI活用：生成AI（言語処理，活用実践）
第8回	総合演習（センサーデータ）

MCJ-CloudHub

■環境

- NII 提供の試運用環境, メモリ 1 GB/人 相当
- 大学 Moodle と LTI 連携
(アカウント自動紐付け, 評点連携は不使用)

■構築・設定

- MCJ-CloudHub サポート :
ライブラリ・パッケージ・環境変数等の追加
- Moodle 管理者 :
LTI 外部ツール設定

自動採点の活用

■ 参考にしたページ

- [Nishii's Notebook](#), [nbgrader official](#)

■ Python と R で同等の課題を作成するための工夫

- 単純計算：直前セルの最終評価値（Python: `_` R: `.Last.value`）または特定変数への代入で評価
- プロット：画像化して比較（Python: `gcf()` R: `.Last.value`）
- 例外生成：Python: `assert` 文, R: `stopifnot` 関数
- 解答セルの表示調整：`nbgrader_config.py` に言語別の設定

■ 課題

- 言語選択式の演習では、50%を満点とする採点が必要

生成 AI の簡易活用

■ ChatGPT API

➤ 環境変数 OPENAI_API_KEY に API キーを設定

➤ Python
openai.OpenAI.client.responses

➤ R
httr2:request |>
req_auth_bearer_token |>
req_body_json

■ 課題

➤ 自動採点時にも API が呼び出されてしまう

```
[ ]: # Vibe coding を実行するための関数を定義
from openai import OpenAI

client = OpenAI()

responses = []
def vibe(prompt, use_previous=True):
    global responses
    if use_previous and responses:
        res_id = responses[-1].id
    else:
        res_id = None

    try:
        instructions = f"""
        You are a helpful coding assistant specialized in data science.
        - Write Python code based on the user's input.
        - Return code only. No explanations.
        - Use print(), plt.show(), etc. explicitly to show results.
        """
        res = client.responses.create(
            model="gpt-5-nano",
            instructions=instructions,
            input=prompt,
            # temperature=0.2, # this gives an error for the gpt-5 models
            store=True,
            previous_response_id=res_id
        )

        responses.append(res)
        code = res.output_text
        print("\n#---- 生成されたコード ----\n")
        print(code, "\n")

    if input("実行しますか? (y/n): ").lower() in ("y", "yes"):
        try:
            print("\n#---- コード実行 ----\n")
            exec(code, globals())
        except Exception as e:
            print(f"#コード実行エラー: {e}")

[ ]: # Vibe coding を実行するための関数を定義
library(httr2)

# 過去 Response を保存
responses <- list()

url <- "https://api.openai.com/v1/responses"

vibe <- function(prompt, use_previous = TRUE) {
  api_key <- Sys.getenv("OPENAI_API_KEY")
  if (api_key == "") {
    stop("環境変数 OPENAI_API_KEY が設定されていません。")
  }

  # 前回のレス ID
  if (use_previous && length(responses) > 0) {
    prev_id <- responses[[length(responses)]]$id
  } else {
    prev_id <- NULL
  }

  instructions <- paste(
    "You are a helpful coding assistant specialized in data science.",
    "- Write R code based on the user's input.",
    "- Return code only. No explanations.",
    "- Use print(), plot(), etc. explicitly to show results.",
    sep = "\n"
  )

  # リクエスト生成
  req <- request(url) |>
  req_auth_bearer_token(api_key) |>
  req_body_json(list(
    model = "gpt-5-nano",
    input = prompt,
    instructions = instructions,
    # temperature = 0.2, # this gives an error for the gpt-5 models
    store = TRUE,
    previous_response_id = prev_id
  ))
}
```

構築・本運用に向けて (2026年度)

背景

■ 目標

- **オンプレサーバ**での MCJ-CloudHub の構築：
北見工業大学（本学と法人統合）GPU サーバを活用
- 学外運用の手續に時間を要し，保留中

■ 予備構築

- **mdx1**：IdP→SPの属性送信設定に時間を要し，保留中
- **AWS**：高額化しやすいが，VCP 対応クラウドで即時利用可

構築概要

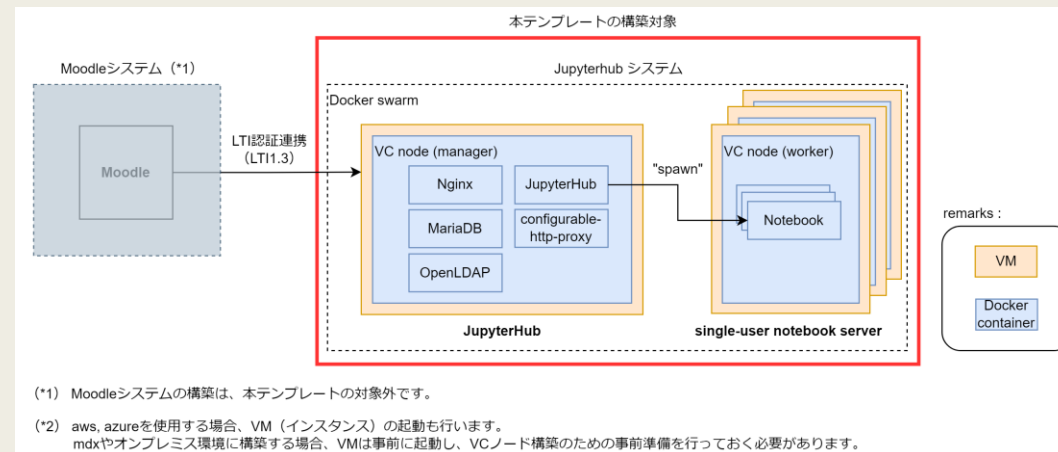
■ 使用インターフェース・技術

- EC2コンソール, Linux CUI, VCP SDK (Jupyter), DNS, Moodleサイト管理
- AI (Claude) との協働によるソース・ログ解析



■ 構築の流れ

- AWS 上に構築環境 VM を用意
(ap-southeast-2, t3.medium, Ubuntu 26.04,
VPN/IPsec 不要)
- 構築環境 に VCP ポータブル版を
レポジトリ取得, VCC/SDK 構築
- SDK に MCJ-CloudHub をレポジトリ取得,
構築
- Let's Encrypt で証明書取得,
Moodle (4.5.3) と LTI 1.3 連携設定



構築作業上で生じた問題

■ AWS利用

- クレジットカード登録の 3DS が通らずはねられる

■ 実行環境

- Ubuntu 26.04 で用意してしまい、スクリプトやパッケージのバージョン調整が必要に

■ セキュリティ設定

- マニュアル記載のポートのみを開けて作業→誤記載（？）・不記載ポートにより VC ノードの起動制御ができず

■ AWS設定

- 予算上限アラートでインスタンスが強制終了し、SDK から manager/worker 復帰できず

構築から得られた示唆

■ 授業者が構築する意義

- 柔軟・機動性の向上, 学習者情報の保護, トラブル対応の易化
- SDK・テンプレートの活用により属人化も回避可能

■ 生成 AI との協働の可能性

- 特に OSS のコードやログの解釈で圧倒的な効果
- 環境・設定依存の現象では人とのやり取りが重要

■ 研究開発への貢献

- 多様な参画・フィードバックを通じた改善・高度化への期待

まとめ

■ 帯広畜産大学の事例から

- MCJ-CloudHub は、大人数プログラミング教育環境として有用

■ 構築試行の事例から

- MCJ-CloudHub は、適切な準備・設定の元で、授業者が AWS で構築可能
- 今後 mdx, オンプレ環境等における構築へ展開